

Sistemas Distribuidos 1 (75.74)

Documento de Arquitectura: IMDb Analyzer
Primer Cuatrimestre de 2025
Grupo 17

Alumno	Padrón	Mail
Joaquin Ariel Pandolfi	108215	jpandolfi@fi.uba.ar
Theo Lijs	109472	tlijs@fi.uba.ar
Francisco Juarez	107748	fjuarez@fi.uba.ar

Índice

1. Introducción	3
2. Vista Escenarios	3
2.1. Requerimientos Funcionales	3
2.2. Requerimientos No Funcionales	3
3. Visión Lógica	4
3.1. Flujo de Datos	4
4. Vista Física	5
4.1. Distribución de Componentes	5
4.2. Sharding en el sistema	6
4.3. Interacciones de Componentes	6
5. Vista de Procesos	9
5.1. Comunicación Cliente-Servidor	9
5.1.1. Múltiples clientes	9
5.2. Mecanismo de Control de Fin de Datos por Sesión de Cliente	10
5.3. Algoritmo de junta	11
5.4. Reducers y Final Reducers	12
5.5. Tolerancia a Duplicados y Mecanismo de Filtrado	12
5.5.1. Algoritmo de filtrado de duplicados por ventana	13
5.6. Tolerancia a fallas	15
5.6.1. Checkpoints	15
5.6.2. Log de operaciones	15
5.6.3. Persistencia atómica en disco	17
5.6.4. Consistencia entre Checkpoints y Logs	19
5.6.5. Balance entre rendimiento y robustez	20
5.7. Monitoreo	20
5.7.1. Healer	20
6. Vista de Desarrollo	23
6.1. División del Sistema	23

7. Limitaciones y posibles mejoras

24

1. Introducción

En el siguiente documento se describe la arquitectura del sistema IMDb Analyzer, cuyo objetivo es el procesamiento de datasets de películas, reviews y actores de forma distribuida para así responder cinco consultas distintas.

2. Vista Escenarios

2.1. Requerimientos Funcionales

El sistema recibe del cliente los datasets de IMDb para luego responder las siguientes consultas:

- Películas y sus géneros de los años 2000 con producción Argentina y Española.
- Top 5 de países que más dinero han invertido en producciones sin colaborar con otros países.
- Película de producción Argentina estrenada a partir del 2000, con mayor y con menor promedio de rating.
- Top 10 de actores con mayor participación en películas de producción Argentina con fecha de estreno posterior al 2000
- Average de la tasa ingreso/presupuesto de películas con overview de sentimiento positivo vs. sentimiento negativo

2.2. Requerimientos No Funcionales

- El sistema debe estar optimizado para entornos multicomputadoras
- Se debe soportar el incremento de los elementos de cómputo para escalar los volúmenes de información a procesar
- Se requiere del desarrollo de un Middleware para abstraer la comunicación basada en grupos.
- Se debe soportar una única ejecución del procesamiento y proveer graceful exit frente a señales SIGTERM.
- Soporte para varias ejecuciones de las consultas para un cliente, sin reinicio del servidor.
- Ejecución con varios clientes concurrentemente.
- Correcta limpieza de los recursos luego de cada ejecución.
- El sistema debe ser tolerante a fallos por caídas de procesos.

3. Visión Lógica

3.1. Flujo de Datos

En la figura 1 se puede ver el DAG, en donde se muestran los datos de input y output en cada paso que hace el sistema para completar las queries.

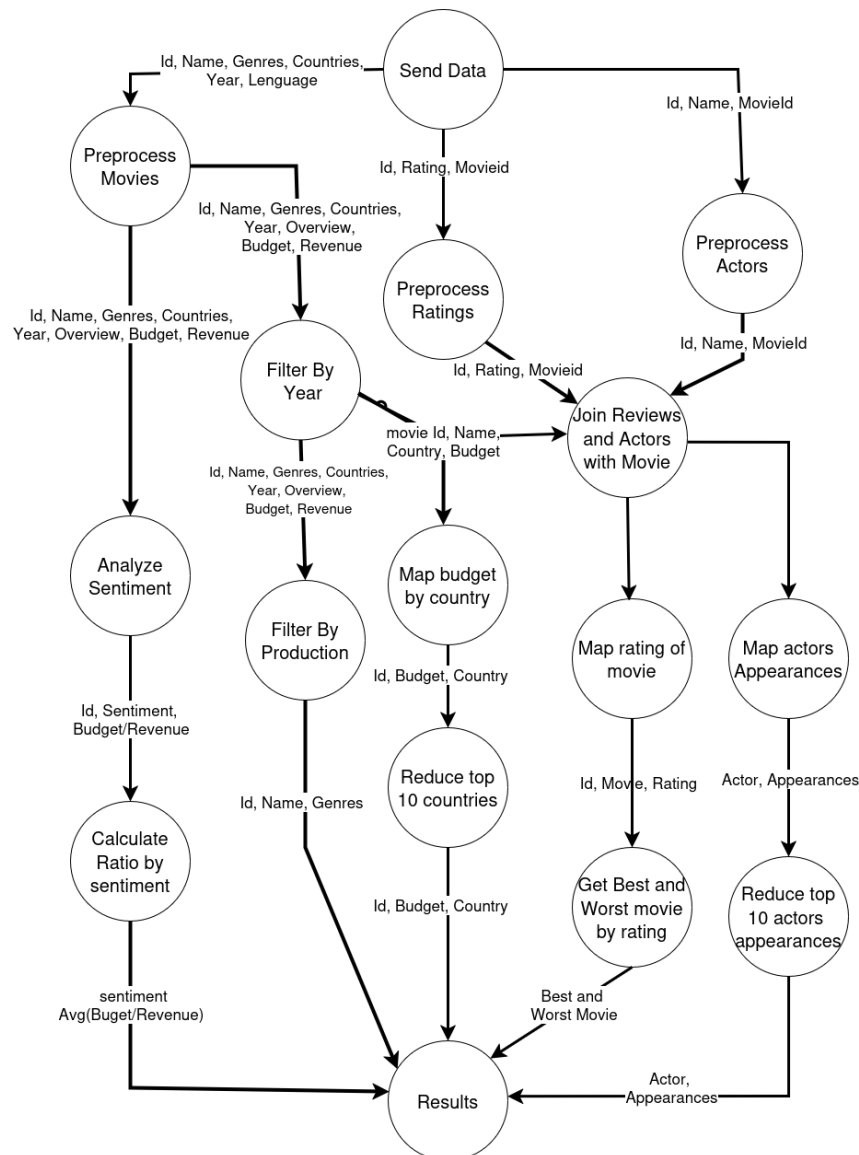


Figura 1: DAG

Cabe aclarar que en el módulo de results, el servidor se encarga de enviar los resultados al cliente una vez finalizado el procesamiento de cada una de las consultas. Además, en aquellos casos donde la naturaleza de la consulta lo permite —por ejemplo, "películas y sus géneros de los años 2000 con producción argentina y española"— el servidor tiene la capacidad de enviar resultados parciales a medida que se van generando, mejorando así la eficiencia y reduciendo la espera por parte del cliente.

4. Vista Física

4.1. Distribución de Componentes

A continuación, la figura 2 muestra la topología del sistema a nivel de vista física y las conexiones entre sus componentes físicos.

Se utiliza RabbitMQ como message broker para la implementación del middleware, que se encarga de casi toda la comunicación dentro del sistema entre los diferentes nodos del mismo.

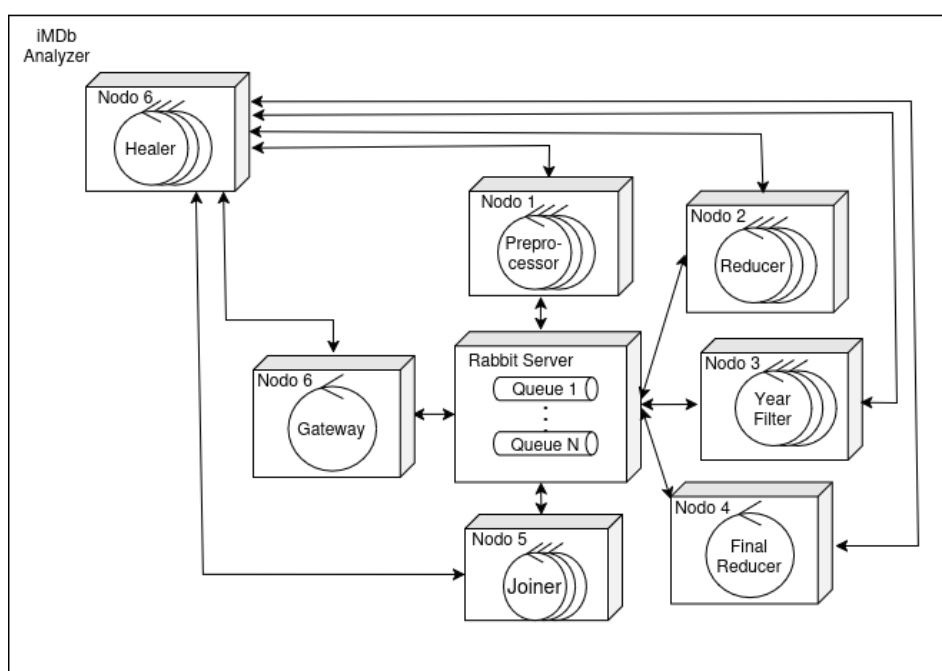


Figura 2: Diagrama de Despliegue

Es importante aclarar que no se requiere que ningún nodo este en la misma computadora. Por ejemplo, en el nodo 1 se podrían desplegar, además de los preprocessors, algunos joiners, sin importar que hayan más de estos en el nodo 5.

Si es importante en la configuración especificar cuantos joiners hay en el sistema a los siguientes nodos: Final Reducers, Preprocessors, Production Filters. Esto es por temas de sharding que sucede en los joiners explicado en la próxima sección.

A su vez, es importante que los healers(monitores) tengan acceso a un archivo con la ip o alias de todos los servicios que deben monitorear y revivir. (incluidos otros healers).

Todos los nodos del sistema se comunican adicionalmente con los nodos de monitoreo (Healers) mediante conexiones TCP directas. Esta comunicación, utilizada exclusivamente para monitoreo y detección de fallos, no modifica la topología principal del despliegue ni interfiere con el flujo de mensajes a través de RabbitMQ. No obstante, requiere que todos los nodos estén accesibles en red desde los Healers, lo cual es un requerimiento compartido también por RabbitMQ para el enrutamiento de datos.

4.2. Sharding en el sistema

Para tolerar la alta carga de procesamiento generada por las queries 3 y 4 -las cuales requieren joins entre películas, ratings/reviews y créditos- se optó por aplicar un esquema de sharding sobre los nodos Joiner.

Cada Joiner es responsable de un shard particular del dominio de datos, y recibe únicamente las películas, ratings/reviews y créditos que le corresponden según dicho shard. El proceso de shardeo se realiza en dos puntos distintos del flujo:

- En el Preprocessor: se realiza el shardeo de ratings, reviews y créditos. La asignación a shards se basa en el `movie_id` presente en cada uno de estos registros.
- En el filtro de producción (Production Filter): se shardean las películas justo antes de ser enviadas a los Joiners, ya que este filtro es el último paso de la pipeline de procesamiento para las queries afectadas.

El criterio de shardeo está basado en un hash del ID de la película: se calcula un hash SHA-256 del `movie_id` (considerado como string), y se toma el primer byte del hash para determinar a qué shard pertenece. La fórmula es la siguiente:

$$\text{shard_id} = (\text{byte}_0(\text{SHA256}(\text{movie_id})) \bmod N) + 1$$

Este mecanismo garantiza una distribución determinista de los datos entre los Joiners. Al basarse en hashing, se evita cualquier dependencia de la estructura interna del ID, y se asegura que todos los registros asociados a una misma película -tanto sus atributos principales como los ratings, reviews y créditos- sean enviados al mismo Joiner. Esto permite que el join se realice de manera local en cada nodo, sin necesidad de replicar información ni realizar coordinaciones adicionales entre shards.

Se aplicó sharding para distribuir la carga entre los Joiners sin duplicar datos. Sin esta estrategia, cada Joiner debería recibir todas las películas, lo que saturaría las colas de RabbitMQ y limitaría la escalabilidad. Al shardear, cada película y sus datos asociados se envían a un único nodo, permitiendo escalar horizontalmente al agregar más Joiners, teniendo en cuenta que implica tener mas sharding.

4.3. Interacciones de Componentes

Seguidamente, en la figura 3, ubicada en la siguiente hoja en orientación horizontal, se muestra la interacción entre los diferentes servicios del sistema con respecto a los datos del mismo.

En el diagrama de robustez se observa el sharding representado mediante los componentes Joiner1 ... JoinerN, que corresponden a instancias paralelas encargadas de procesar en forma distribuida los distintos shards de datos. Cada Joiner recibe únicamente las películas y registros asociados que le corresponden según el particionamiento previamente descrito.

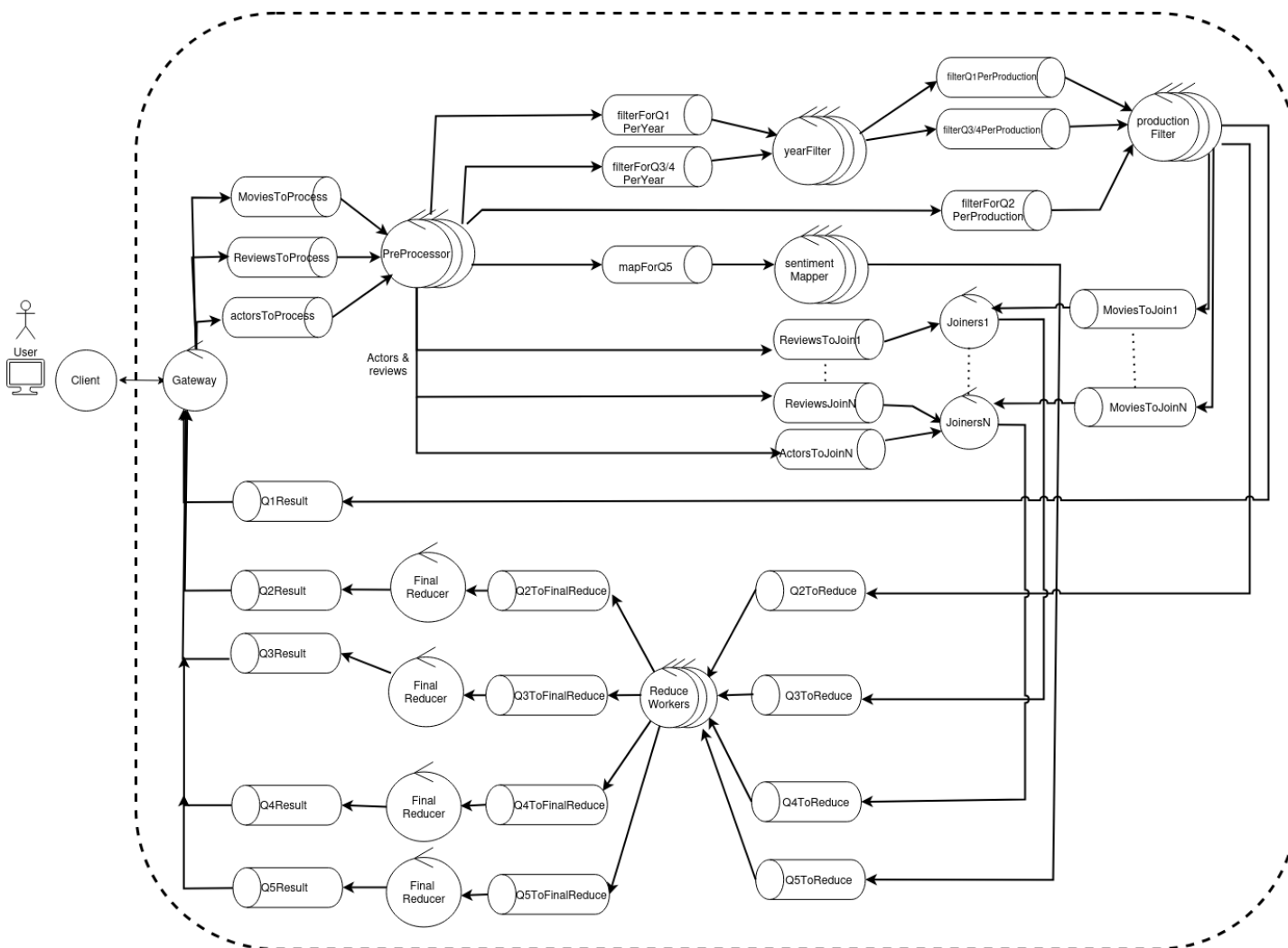


Figura 3: Diagrama de Robustez

Para ilustrar el comportamiento específico del sistema frente a la caída de un cliente, se incluye a continuación un diagrama de robustez parcial que muestra la interacción entre el Gateway, los Joiners y los Final Reducers mediante una cola de control dedicada al mecanismo de flush.

Es importante recalcar que los nodos que consumen de la FlushQueue son los nodos que manejan un estado interno, ya que estos al recibir el mensaje borran la data correspondiente al cliente.

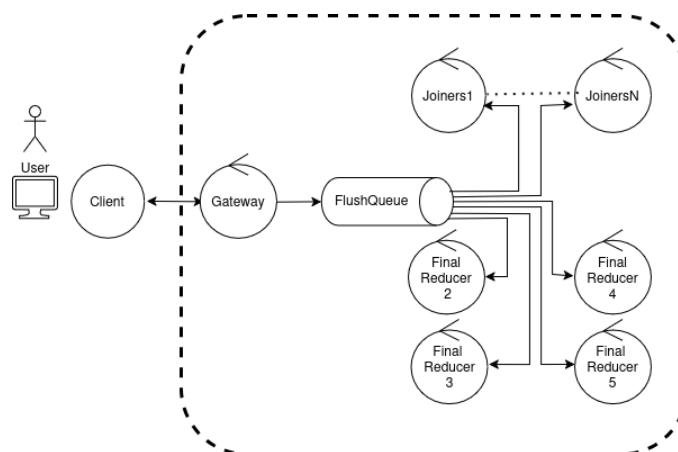


Figura 4: Diagrama de robustez control de flush

Ante la desconexión de un cliente, el Gateway detecta el evento y publica un mensaje en la FlushQueue que contiene el UUID del cliente. Esta cola, de tipo fanout, distribuye el mensaje a todos los nodos suscritos. Cada nodo que mantiene estado interno procesa dicho mensaje eliminando la información asociada al cliente y registrando su UUID en una blacklist interna. Esta lista negra permite filtrar cualquier mensaje futuro proveniente del cliente desconectado, evitando así el uso innecesario de recursos de cómputo.

Adicionalmente, el Gateway persiste una lista actualizada de clientes activos en un archivo local. Esta lista se modifica dinámicamente ante cada conexión o desconexión de clientes. Su propósito principal es permitir la recuperación del estado en caso de una falla del Gateway. Al reiniciarse, y dado que no puede reconectarse directamente con los clientes, el Gateway utiliza este archivo para emitir mensajes de flush dirigidos a todos los clientes previamente registrados, asegurando la limpieza del estado distribuido y manteniendo la coherencia global del sistema.

5. Vista de Procesos

5.1. Comunicación Cliente-Servidor

En el diagrama de secuencia de la figura 5 se puede observar el flujo de mensajes que se intercambian al realizar la comunicación entre el cliente y el sistema consultas. Se observa el inicio de la comunicación y la recepción de los datasets por parte del servidor, luego se abstrae en este diagrama la resolución de las consultas y por último se puede ver la devolución de los resultados obtenidos.

Es importante mencionar que, al finalizar el envío de cada dataset, el cliente envía un marcador de fin de archivo (EOF) correspondiente. Este EOF se propaga por el sistema y es fundamental para los componentes que requieren conocer cuándo se ha completado el ingreso de datos, como es el caso de los Joiners, que deben esperar a tener todas las películas necesarias antes de poder continuar con el proceso de join.

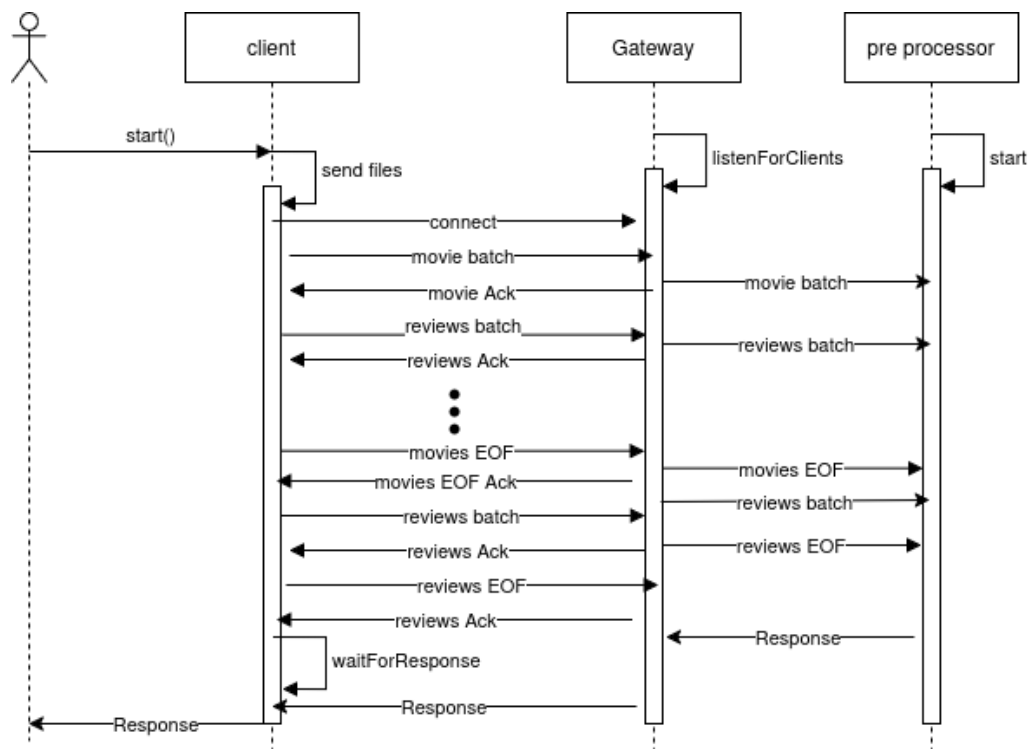


Figura 5: Diagrama de Secuencia Cliente - Servidor

5.1.1. Múltiples clientes

Para manejar múltiples clientes de forma simultánea, el gateway asigna un identificador de sesión único (UUID) a cada batch de datos. Este identificador permite a los distintos componentes del sistema -como los final reducers y los joiners- mantener aislados los datos de cada cliente. De este modo, se facilita una gestión independiente de la información, incluyendo la posibilidad de eliminar los datos de cada sesión de manera individual.

5.2. Mecanismo de Control de Fin de Datos por Sesión de Cliente

Para asegurar que todos los datos enviados por los clientes sean correctamente procesados y permitir operaciones que dependen de la totalidad de los datos (como joins o reducciones finales), se implementa un esquema de control basado en metadatos transportados en los headers de cada batch.

Cada batch transporta dos campos clave en su encabezado:

- **weight**: Indica el tamaño o peso del batch actual.
- **total_weight**: Representa el peso total esperado de todos los batches correspondientes a la sesión de ese cliente.

Por diseño, durante la transmisión normal de datos, el campo **weight** tiene el valor del tamaño del batch actual y el campo **total_weight** tiene el valor **-1**. Este último valor se actualiza únicamente en el último batch de la sesión, donde se informa el peso total acumulado de la misma. Este último batch actúa como señal de fin de datos (EOF).

Es importante destacar, que el *weight* no representa una cantidad de filas o bytes en el mensaje, sino la proporción del dataset original que se está representando. Por más que se filtren filas, el peso del batch seguirá siendo el mismo, y al juntar con otro, el peso será igual a la suma de las partes. Esto permite a los nodos saber si toda la información necesaria pasó por ese punto o si aún debe esperar.

Es así entonces, que los nodos que requieren disponer de la totalidad de los datos para su correcto funcionamiento, como los *Joiners* y *Final Reducers*, aplican la siguiente lógica.

1. Contadores por Sesión:

Cada nodo mantiene un contador independiente de **weight** recibido para cada sesión de cliente activa.

2. Acumulación de Datos:

Con cada batch recibido, el nodo incrementa el contador correspondiente sumando el valor de **weight**.

3. Recepción de EOF:

Al recibir un batch cuyo campo **total_weight** es distinto de **-1**, el nodo interpreta que es el último batch de la sesión y conoce el peso total esperado.

4. Verificación de Completitud:

Si el peso acumulado coincide con el **total_weight**, el nodo considera que ha recibido todos los datos y puede continuar (ejecutar el join, reducir los datos, o liberar el resultado final). Por otro lado, si los pesos no coinciden, el nodo permanece a la espera de los batches faltantes, dado que los mensajes pueden llegar desordenados o con retraso.

Este mecanismo permite tolerar la llegada desordenada de mensajes sin comprometer la integridad de los datos procesados.

5.3. Algoritmo de junta

Para las queries 3 y 4, el sistema debe realizar una junta entre dos de las tablas. Para esto, el sistema lo interpreta como una operación sobre los datasets *reviews.csv* y *credits.csv*.

En este proceso, al bloque encargado de la junta (el joiner) le pueden suceder dos eventos distintos. El primero es que le llegue un batch de películas, el segundo es que le llegue un batch de otro de los datasets.

Es posible que los *joiners* no puedan resolver la tarea debido a que no recibieron todas las películas transformadas correspondientes. En este caso, los *joiner* la almacenaran internamente y una vez que llegue la ultima película, procesara todas las reviews pendientes antes de continuar.

Una vez que se terminan de juntar todas las películas con las reviews y los credits, se flushean/-borran las películas del cliente correspondiente.

En la siguiente figura 6 se puede observar el flujo explicado:

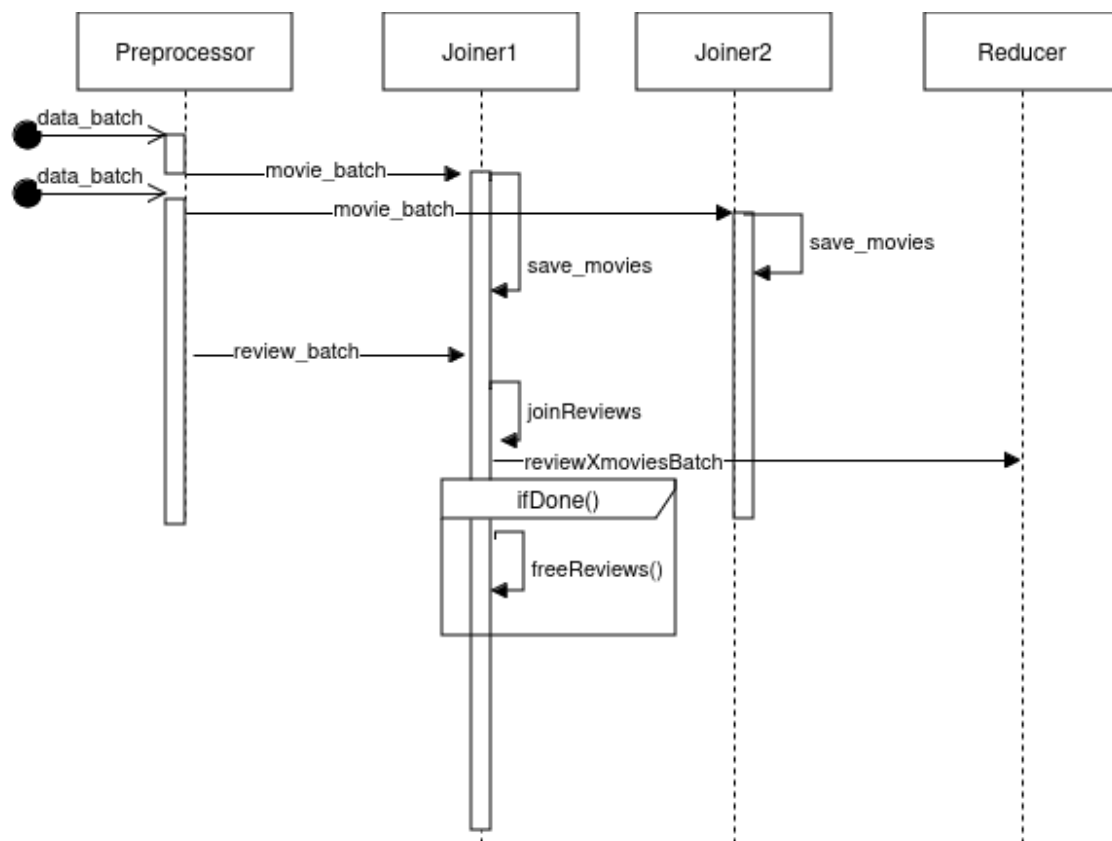


Figura 6: Diagrama de Secuencia de Joiner

5.4. Reducers y Final Reducers

Los *Reducers* del sistema conforman un *pool* de *Workers* sin estado (*stateless*), capaces de procesar tareas de cualquier tipo enviadas por otros nodos del sistema. Una vez resueltas, los resultados son enviados al *Final Reducer* correspondiente. Para cada consulta (*query*) que requiera acumulación de resultados, existe un nodo *Final Reducer* dedicado. Este nodo se encarga de recolectar los resultados parciales enviados por los *Reducers* y, al recibir el EOF, ejecuta la operación final sobre los datos acumulados y devuelve el resultado al *Gateway*.

A continuación se puede observar en la figura 7 el funcionamiento y sincronización del final reducers y los joiners:

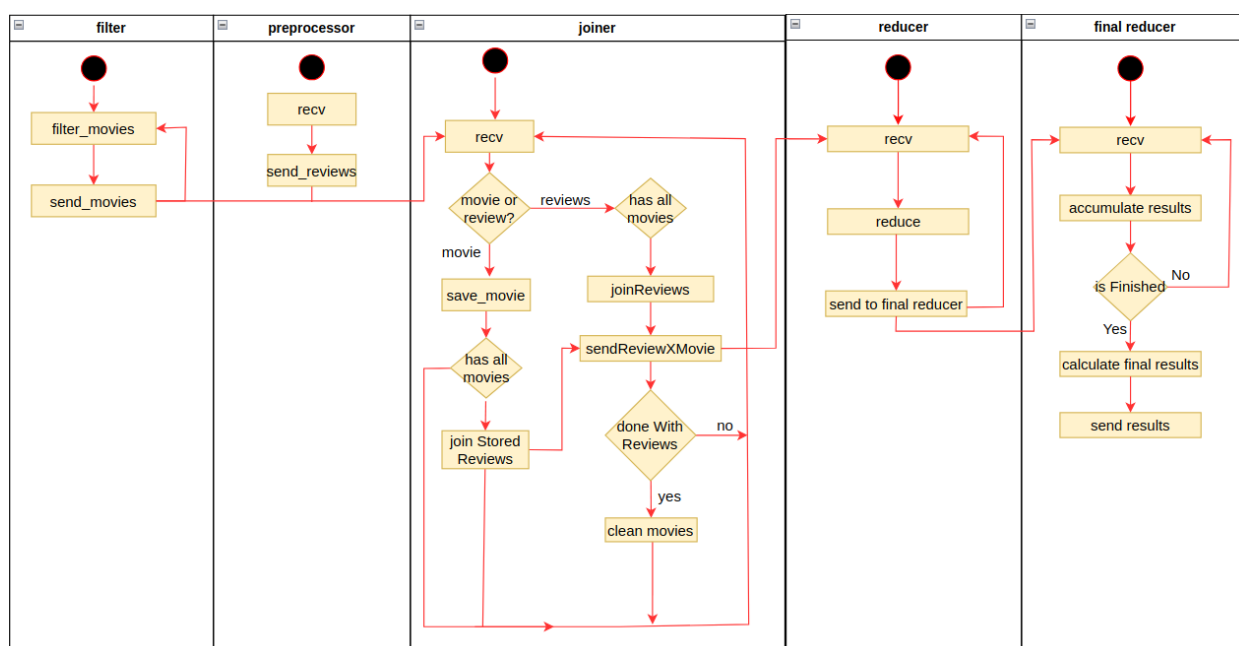


Figura 7: Diagrama de Actividades Reducers-Joiners

Cabe destacar que en el diagrama anterior, el nodo identificado como *filter* representa a los nodos *year filter* y *production filter*. Estos componentes implementan una lógica sencilla: reciben batches de películas desde las Queues correspondientes a distintas Queries, aplican el filtro específico según su la query, y envían los resultados filtrados a la siguiente queue del flujo de procesamiento.

5.5. Tolerancia a Duplicados y Mecanismo de Filtrado

En sistemas distribuidos basados en colas de mensajería como RabbitMQ, existe la posibilidad de que se generen mensajes duplicados debido a fallos en los nodos emisores. Un escenario común ocurre cuando un nodo envía un mensaje, pero falla antes de poder confirmar su correcta recepción (ACK). Al reiniciarse el nodo, la cola de mensajería le reenvía el mensaje a este, ya que nunca recibió un ACK de su parte. Por ende el nodo vuelve a procesar y enviar el mismo mensaje, generando así un duplicado en el sistema.

Para evitar que estos duplicados provoquen inconsistencias o procesamiento incorrecto, se implementa un mecanismo de filtrado de duplicados en los siguientes componentes críticos del sistema: *Joiners*, *Final Reducers* y el *Gateway*.

El mecanismo se basa en la asignación de un identificador incremental único a cada batch, gestionado por el Gateway. Este identificador presenta las siguientes características:

- Es incremental y único por cliente.
- Se mantiene de forma independiente por tipo de dato (películas, reviews, créditos).

De esta forma, cada flujo de datos de un cliente posee su propia secuencia de identificadores, evitando colisiones entre diferentes tipos de datos o sesiones.

Cuando un batch alcanza, por ejemplo, un Joiner, el nodo verifica el identificador asociado:

- Si el identificador corresponde a un mensaje nuevo, el batch se procesa normalmente.
- Si el identificador ya fue procesado previamente, el batch se descarta de manera segura.

Este mecanismo garantiza que, incluso si se producen duplicados debido a fallos y reinicios de los nodos emisores, el sistema permanece consistente y libre de errores asociados a la duplicación de datos.

Un caso particular se presenta en los *Final Reducers*, ya que debido al esquema de *sharding*, no solo se debe considerar el identificador del batch, sino también el identificador del shard correspondiente desde el cual proviene el batch procesado por los *Joiners*. Esto implica que los *Final Reducers* deben mantener ventanas de control independientes por shard. Esto pasa ya que cuando el preprocessor realiza el shardeo de reviews por ejemplo, el mismo batch puede ser dividido entre uno o mas shards dependiendo el ids de las películas.

5.5.1. Algoritmo de filtrado de duplicados por ventana

El algoritmo gestiona una ventana ordenada de IDs de mensajes que ya fueron recibidos. Cuando llega un nuevo ID, se inserta en la posición correspondiente dentro de la ventana y posteriormente se eliminan los ids desde el primero de la ventana hasta el primero faltante. Si el ID ya estaba presente o es menor al mínimo de la ventana actual, se descarta porque ya fue procesado previamente.

Este enfoque evita almacenar los IDs de todos los mensajes recibidos, ya que al compactar los consecutivos solo se conserva el primer ID faltante y posibles gaps aislados. Aunque los mensajes pueden llegar desordenados, en condiciones normales la ventana se mantiene pequeña y la cantidad de IDs que deben recordarse es baja.

En el caso del Final Reducer, donde los datos se procesan en paralelo por shard, se mantiene un filtro de duplicados independiente por cada shard, aplicando la misma lógica de ventanas para cada uno de forma aislada.

A continuación en la figura 8 se puede ver un caso muy simple del algoritmo en acción:

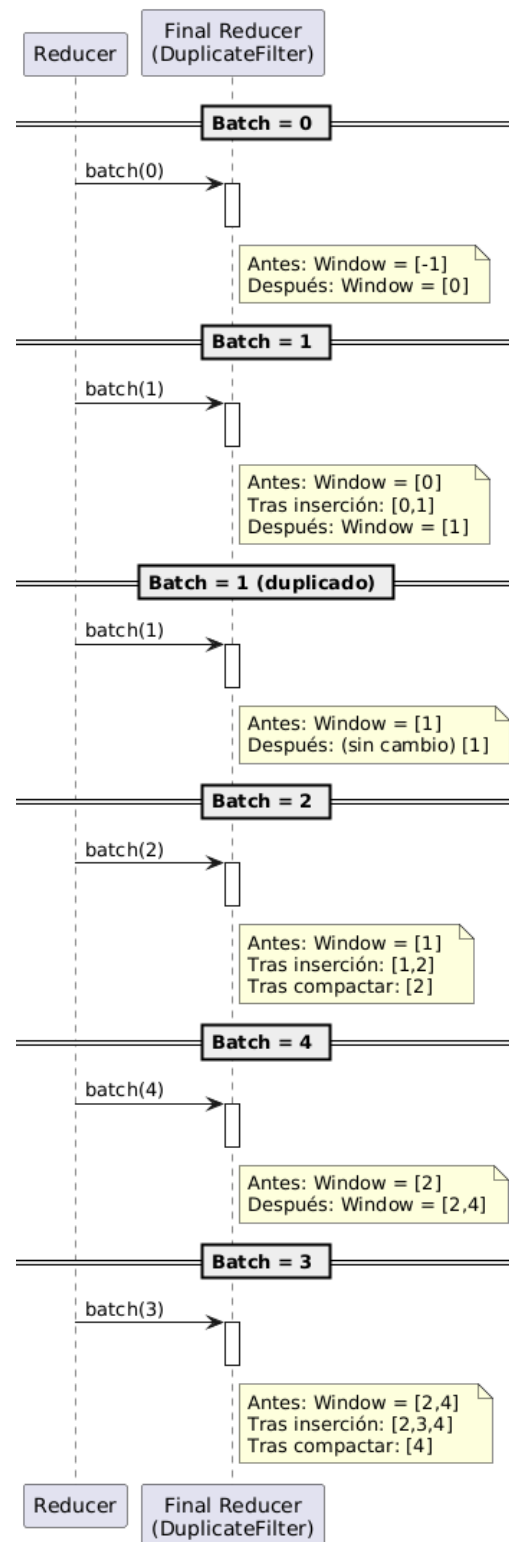


Figura 8: Diagrama de secuencias simplificado Reducers-Final Reducers

5.6. Tolerancia a fallas

Uno de los requisitos más desafiantes e interesantes en el diseño de sistemas distribuidos es la capacidad de tolerancia a fallas. Este concepto implica que el sistema debe ser capaz de continuar funcionando o, al menos, recuperarse adecuadamente ante fallos inesperados, incluyendo los fallos fatales como la terminación abrupta del proceso (por ejemplo, mediante una señal SIGKILL).

La tolerancia a fallas es fundamental en sistemas distribuidos ya que estos suelen estar compuestos por múltiples procesos que se ejecutan en distintos nodos, cada uno sujeto a sus propios posibles errores de red, hardware o software. En nuestro caso, implementamos un enfoque híbrido basado en la persistencia periódica del estado del sistema mediante **Checkpoints** y la trazabilidad de las acciones mediante un **Log de operaciones**. Esta combinación permite alcanzar un equilibrio entre eficiencia, simplicidad y robustez frente a fallas.

5.6.1. Checkpoints

Un **Checkpoint** representa el estado interno del sistema en un momento dado. Se almacena en una forma persistente y recuperable, permitiendo restaurar el sistema en caso de una interrupción catastrófica.

La creación de Checkpoints reduce significativamente el tiempo necesario para recuperar el sistema, ya que permite tener guardado el estado previamente calculado y al levantarse el sistema lo carga a memoria. De esta forma la recuperación es muy rápida y el sistema puede empezar a procesar mensajes de vuelta.

El sistema realiza Checkpoints de forma esporádica, ya que al ser una operación lenta, hacerlo constantemente perjudica fuertemente la velocidad del sistema.

Formato de almacenamiento. Actualmente, el Checkpoint se representa como un archivo JSON que contiene una serialización completa del estado del sistema. Esta elección conlleva varias ventajas:

- **Facilidad de persistencia:** JSON es un formato ampliamente soportado, lo que permite guardarlo en disco sin necesidad de diseñar un formato binario personalizado.
- **Legibilidad:** el formato textual facilita la inspección manual del estado, útil para tareas de depuración.
- **Portabilidad:** al ser un estándar de facto, JSON permite mayor interoperabilidad entre distintos sistemas o versiones.

No obstante, también presenta algunas desventajas:

- **Tamaño:** al ser un formato textual, el tamaño del archivo es mayor en comparación con alternativas binarias.
- **Velocidad:** la serialización y deserialización de JSON es generalmente más lenta, especialmente para estructuras de datos complejas o grandes volúmenes de información.

5.6.2. Log de operaciones

Complementariamente al mecanismo de Checkpoints, el sistema mantiene un **Log de operaciones** que registra cada acción relevante ejecutada desde el último Checkpoint. Este Log actúa como una

bitácora detallada que permite reconstruir el estado exacto del sistema, incluso si este ha fallado justo antes de realizar un nuevo Checkpoint.

Granularidad del Log. Las entradas del Log corresponden a operaciones determinísticas que afectan el estado interno del sistema. Estas operaciones son registradas línea por línea, en un formato que permite su reejecución secuencial y fiel durante el proceso de recuperación. Cada entrada contiene suficiente información como para ser replicada sin ambigüedades, incluso si se ejecuta en un contexto distinto al original.

Modelo transaccional. Cada vez que el sistema recibe un mensaje desde el middleware, se inicia la construcción de una **transacción en memoria**. Esta transacción es una estructura que agrupa todas las operaciones relevantes que se realizarán como consecuencia del procesamiento del mensaje. Por ejemplo, si el mensaje implica modificar estructuras internas, agregar películas a algún cliente, actualizar un contador del final reducer, todas estas acciones se representan en forma de operaciones registrables.

Estas operaciones no se aplican directamente al Log una por una. En cambio, se acumulan en la transacción en memoria. Una vez finalizado el procesamiento del mensaje —pero antes de enviar el ACK al middleware—, el sistema **serializa la transacción completa en el Log** en una única operación. Cada operación de la transacción se escribe en una línea, y al final se añade una línea adicional con un **carácter de commit especial** (por ejemplo, 'c' o cualquier marcador definido) que indica que la transacción fue completada exitosamente.

Confirmación de persistencia antes del ACK. El ACK al middleware se envía únicamente una vez que la transacción ha sido completamente registrada en el Log y el mensaje se ha enviado exitosamente a las colas correspondientes. Esto garantiza que ningún mensaje será considerado "procesado" sin que su efecto haya sido persistido correctamente. Si se produce una falla antes de escribir completamente la transacción o su marca de commit, el sistema no enviará el ACK, y por lo tanto, el mensaje será reenviado automáticamente por el middleware tras la recuperación.

Detección de transacciones incompletas. Durante el proceso de recuperación, tras la carga del último Checkpoint, el sistema reanaliza secuencialmente el Log desde ese punto. En esta fase, sólo se aplican aquellas transacciones que tienen correctamente escrita su línea de commit al final. Las transacciones que no contienen esta marca se consideran incompletas o corruptas (por ejemplo, porque la escritura fue interrumpida por una falla repentina). Estas transacciones son descartadas completamente ya que sabemos que no se hizo ACK al mensaje y el middleware nos lo va a reenviar.

Este comportamiento es seguro gracias al modelo de entrega garantizada del middleware (at-least-once delivery en RabbitMQ), que asegura que los mensajes que no hayan sido confirmados con un ACK serán reenviados tras la recuperación. De esta forma, el sistema evita procesar parcialmente transacciones incompletas y garantiza consistencia sin duplicación indebida de efectos.

Ejemplo detallado de transacciones en el Log. A continuación se muestra un ejemplo realista de dos transacciones consecutivas registradas en el Log del Final Reducer de la Query 3.

```
agg_movie_ratings {"header":{"weight":5120,"total_weight":-1,"client_id":"e85072bc-a713-41c2-8748-d80321474939","message_id":{"id":5031,"joiner_id":4}},"data":[]}  
filter_duplicates e85072bc-a713-41c2-8748-d80321474939;5031;4  
update_weights e85072bc-a713-41c2-8748-d80321474939,5120,-1  
c
```

```
agg_movie_ratings {"header":{"weight":5120,"total_weight":-1,"client_id":"43c96101-7720-445a-85f9-f73a15301178","message_id":{"id":5022,"joiner_id":4}},"data":[]}  
filter_duplicates 43c96101-7720-445a-85f9-f73a15301178;5022;4  
update_weights 43c96101-7720-445a-85f9-f73a15301178,5120,-1  
c
```

Cada transacción en el log puede interpretarse de la siguiente manera:

- **agg_movie_ratings**: esta operación agrega datos de puntuación de películas. Contiene un JSON serializado con metadatos del mensaje, incluyendo peso parcial, identificador del cliente, ID del mensaje y del componente de join.
- **filter_duplicates**: actualiza la ventana de filtrado mensajes que ya hayan sido procesados previamente, usando como clave un identificador compuesto: `client_id;message_id;joiner_id`.
- **update_weights**: actualiza los pesos acumulados para el cliente, en caso del mensaje ser EOF, también lo actualiza. Usa como entrada los mismos identificadores junto a los pesos recibidos.
- **c**: esta línea indica que la transacción fue registrada completamente y de forma segura. Actúa como delimitador y confirmación de persistencia.

Durante la fase de recuperación, el sistema procesa el Log desde el último Checkpoint. Lee cada línea secuencialmente y agrupa las operaciones hasta encontrar un `c`. Sólo esas transacciones completas son reejecutadas.

Si se produjera una falla justo después de escribir la tercera operación de una transacción, pero antes de escribir el carácter `c`, entonces esa transacción sería ignorada completamente en la recuperación. Gracias a que no se envió el ACK al middleware, el mensaje original será reenviado, y su procesamiento será realizado nuevamente desde cero.

Este esquema asegura que el estado del sistema sea **idempotente y consistente**, incluso en presencia de fallas en medio de la escritura de una transacción.

Beneficios del enfoque transaccional. Este modelo de logging presenta múltiples ventajas:

- **Atomicidad de procesamiento**: las operaciones se agrupan y persisten de forma atómica, asegurando que nunca se apliquen efectos parciales.
- **Robustez ante fallos de escritura**: las transacciones incompletas son ignoradas de forma segura.
- **Compatibilidad con modelos de reentrega**: al coordinar los envíos de mensajes, el ACK y la escritura persistente, se mantiene integridad sin riesgo de duplicaciones o inconsistencias.
- **Simplicidad de recuperación**: el Log puede procesarse de forma secuencial, sin requerir estructuras de control complejas.

5.6.3. Persistencia atómica en disco

Un aspecto fundamental para garantizar la resiliencia del sistema ante fallos es asegurar que las operaciones de escritura en disco sean realizadas de forma atómica. Esto es especialmente importante en el contexto de la resiliencia a fallos ya que una escritura incompleta, interrumpida por una falla del sistema o corte de energía, podría dejar archivos en estados corruptos o inconsistentes, comprometiendo así el proceso de recuperación.

Problema de las escrituras parciales. Los sistemas de archivos no garantizan que una llamada a `write()` o incluso `close()` implique que todos los datos estén efectivamente almacenados de manera persistente. En muchos casos, los datos se mantienen en buffers de sistema operativo, o el sistema de archivos puede dejar un archivo parcialmente sobrescrito si ocurre un fallo durante el proceso de escritura. Esta situación es especialmente crítica cuando se trata de sobrescribir archivos que contienen información esencial del estado del sistema, como los logs o los checkpoints.

Solución: escritura atómica con archivo temporal y renombrado. Para evitar este tipo de inconsistencias, el sistema implementa un mecanismo de escritura atómica basado en un patrón ampliamente utilizado: escritura a archivo temporal seguida de renombrado. Este proceso consta de varios pasos clave:

1. **Creación de archivo temporal:** Se crea un archivo temporal en el mismo directorio que el destino final. Esto es importante para que la operación de renombrado posterior pueda ser atómica (la mayoría de los sistemas de archivos garantizan la atomicidad de `rename()` dentro del mismo volumen).
2. **Escritura de datos:** Se escriben todos los datos necesarios en el archivo temporal, en una única operación secuencial.
3. **Sincronización (`fsync`):** Se invoca la operación `fsync` sobre el archivo temporal para forzar la escritura efectiva de los datos al disco físico, asegurando que no permanezcan en buffers del sistema operativo.
4. **Cierre seguro:** Se cierra el archivo, liberando recursos y completando la transacción de escritura.
5. **Renombrado atómico:** Finalmente, se renombra el archivo temporal al nombre definitivo. Esta operación es atómica en la mayoría de los sistemas de archivos modernos, lo que significa que para cualquier proceso externo, el archivo aparecerá de forma completa o no aparecerá en absoluto.

Garantías obtenidas. Este mecanismo proporciona las siguientes garantías fundamentales:

- **Integridad en presencia de fallas:** Si el sistema falla en cualquier punto antes del renombrado, el archivo original permanece intacto. En el peor de los casos, queda un archivo temporal obsoleto, que es limpiado en la siguiente ejecución.
- **Atomicidad desde el punto de vista del lector:** Otros procesos del sistema (o el mismo sistema en recuperación) nunca ven una versión intermedia del archivo. Siempre acceden a una versión completa y válida.
- **Compatibilidad con recuperación confiable:** Al garantizar que el Log de operaciones o los Checkpoints sean persistidos completamente antes de ser utilizados por el sistema, se evita el riesgo de procesar información corrupta o inconsistente.

Distinción entre persistencia atómica en Checkpoints y Logs. La persistencia atómica mediante escritura a archivo temporal y renombrado es indispensable en el caso de los *Checkpoints*, ya que representan un estado completo y único del sistema. Si se corrompe o queda incompleto, no es posible garantizar una recuperación coherente, dado que el sistema se basa en restaurar ese estado como base para aplicar las operaciones del Log. Por eso, los Checkpoints se escriben con la estrategia de escritura atómica previamente explicada.

En contraste, el *Log de operaciones* es inherentemente tolerante a errores parciales, ya que cada transacción escrita incluye un delimitador explícito de *commit*. En caso de una falla durante la escritura, las transacciones incompletas o truncadas simplemente son descartadas durante la recuperación, y el sistema espera volver a recibir y reprocesar el mensaje correspondiente. Para asegurar durabilidad sin sacrificar rendimiento, el Log utiliza un mecanismo denominado *atomic append*, que garantiza que cada operación se escribe secuencialmente y se sincroniza con disco usando *fsync*, sin necesidad de sobrescribir el archivo completo en cada operación. Esto logra un equilibrio entre resiliencia ante fallos y eficiencia de escritura.

5.6.4. Consistencia entre Checkpoints y Logs

En sistemas resilientes a fallos, es común combinar dos mecanismos de persistencia: Checkpoints (que capturan el estado completo del sistema en un momento dado) y Logs (que registran operaciones incrementales desde el último Checkpoint). Para mantener la integridad del sistema durante la rotación de estos mecanismos, es fundamental asegurar una relación consistente entre ambos. Particularmente, el sistema debe garantizar que si se ha creado un Checkpoint a partir de los Logs, estos ahora estén vacíos, ya que la información está en el nuevo Checkpoint.

Problema de consistencia en rotaciones. Supongamos que el sistema decide tomar un nuevo Checkpoint y, como consecuencia, desea vaciar el Log. Este proceso involucra múltiples pasos que, si se ejecutaran de forma descuidada (por ejemplo, simplemente sobrescribiendo los archivos), podrían dejar el sistema en un estado inconsistente si ocurre una falla en medio de la operación. Por ejemplo, el sistema podría:

- Escribir un nuevo Checkpoint válido en disco.
- Fallar antes de vaciar el Log.
- Al reiniciar, asumir erróneamente que el Log debe reejecutarse desde ese nuevo Checkpoint, cuando en realidad aún contiene operaciones basadas en el anterior.

Solución: punteros de Logs y Checkpoints con escritura atómica. Para evitar este tipo de inconsistencias, el sistema sigue una estrategia cuidadosamente ordenada que aprovecha la escritura atómica a disco descrita previamente:

1. **Estrategia de versionado.** Los Checkpoints se almacenan con nombres versionados, que oscilan entre: `checkpoint-0.json` y `checkpoint-1.json`. Esto evita sobrescribir Checkpoints válidos hasta que se haya confirmado la validez del nuevo. Cabe destacar que solamente se usan 0 y 1, no hace falta tener mas versiones.
2. **Escritura del nuevo Checkpoint.** Se crea un nuevo archivo de Checkpoint (ej., `checkpoint-1.json`) usando escritura atómica. Este paso no modifica todavía ninguna referencia activa al nuevo archivo.
3. **Vaciado del Log.** Una vez que el nuevo Checkpoint está confirmado y persistido en disco, se sobrescribe el Log de operaciones de forma atómica, dejando la primera línea del archivo de Log con el nombre del archivo de Checkpoint asociado (por ejemplo, `checkpoint-1.json`).
4. **Referencias consistentes.** Sólo si ambas operaciones (Checkpoint y vaciado del Log) se completan correctamente, el sistema pasa a utilizar el nuevo par Checkpoint-Log. Si el sistema falla antes de completar ambos pasos, al reiniciarse encontrará un Log que aún apunta al Checkpoint anterior (`checkpoint-0.json` siguiendo el ejemplo anterior) y utilizará esa combinación consistente.

Ventajas de este enfoque.

- **Resistencia a fallos en medio de la rotación:** si el sistema cae después de escribir el nuevo Checkpoint pero antes de actualizar correctamente el Log, seguirá utilizando el Checkpoint anterior, ya que el Log no habrá sido vaciado y su puntero interno indicará que se debe cargar el `checkpoint-0.json`.
- **Evita referencias inconsistentes:** nunca se produce una situación en la que el sistema intente reproducir operaciones desde un Log que no es compatible con el Checkpoint que ha cargado.
- **Recuperación segura:** en el peor de los casos, el sistema rehace operaciones desde un estado anterior completamente válido, sin nunca entrar en estados corruptos o no reproducibles.

5.6.5. Balance entre rendimiento y robustez

El diseño del sistema busca un compromiso balanceado entre rendimiento y robustez:

- Si se realizan Checkpoints con demasiada frecuencia, se introduce una sobrecarga de I/O que degrada el rendimiento general.
- Si se depende exclusivamente del Log, el tiempo de recuperación tras una falla puede ser excesivo si el Log es demasiado extenso.

Por lo tanto, se adopta un enfoque mixto donde los Checkpoints son relativamente infrecuentes pero estratégicamente ubicados, mientras que el Log garantiza una trazabilidad completa entre Checkpoints.

5.7. Monitoreo

Para lograr resiliencia ante fallos, es esencial detectar cuando un nodo se cae e iniciar de inmediato la acción que lo ponga nuevamente en línea. Por ello, nuestra solución se compone de dos elementos: un nodo dedicado al monitoreo y la recuperación (*healer*) y un mecanismo para validar la salud de cada nodo (*health-check*).

5.7.1. Healer

El *healer* es el proceso responsable de detectar caídas y resucitar los nodos afectados. Este servicio también debe ser resiliente: aunque no mantenga estado interno, es crucial que, si se detiene, otra instancia pueda ocupar su lugar, reactivar a la instancia original y reiniciar cualquier nodo que haya fallado.

El aumento de instancias garantiza disponibilidad, pero requiere coordinación: los nodos no deben ejecutar simultáneamente la misma tarea de resurrección. Si varios procesos disparan la misma acción, un contenedor podría quedar atrapado en un ciclo de reinicios constantes.

El *healer* se compone de tres secciones principales:

- **Resurrección/Healing:** implementada con Docker-in-Docker, detecta nodos caídos y los pone nuevamente en marcha.

- **Elección de líder:** detecta la ausencia del líder actual y promueve a un worker para que lo reemplace.
- **Capa de comunicación:** provee la interfaz entre nodos y gestiona las conexiones.

Para la elección de líder se implementó el algoritmo bully, modelado como un autómata finito determinista. Los estados y transiciones de cada nodo se corresponden a la figura 9. El estado *worker* corresponde a un nodo de respaldo mientras que el estado *master* a un nodo que se encuentra realizando la tarea de monitoreo. Los estados *candidate* y *non-candidate* en cambio, corresponden a un nodo que sabe que hay una eleccion en curso.

Si bien en el diagrama se muestran los estados y las transiciones principales; se omitieron los estados redundantes que, no obstante, permiten modelar el conjunto como un autómata finito determinista. Es importante notar que el tiempo designado para declarar que llego un timeout varia con el estado. Estos tiempos deben tener en cuenta el tiempo para para conectividad entre nodos (aumenta con la cantidad de healers) y la latencia de red (en un entorno local es baja esa latencia. pero en la red aumentaría considerablemente).

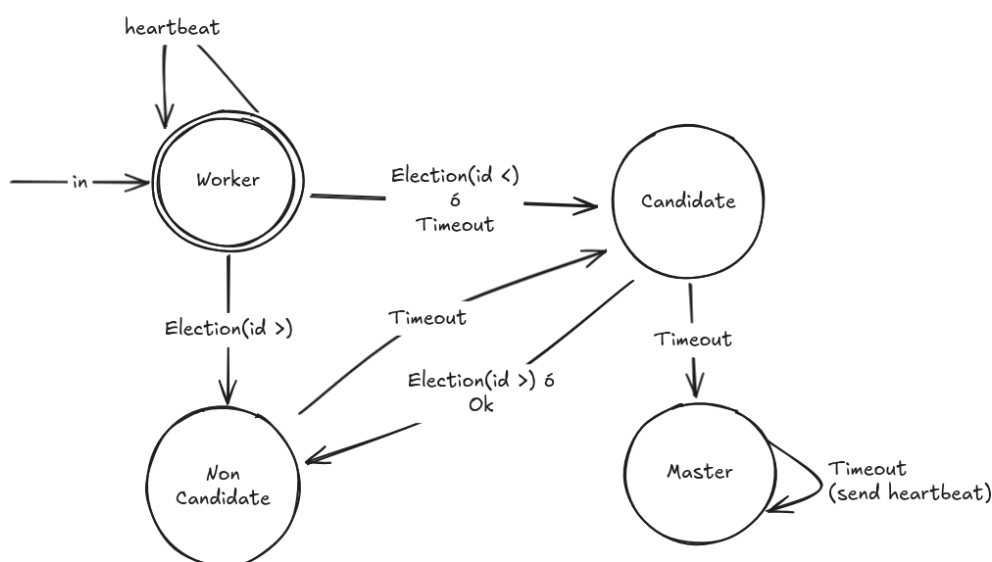


Figura 9: Automata del Algoritmo "Bully"

En la figura 10, se puede observar la posible evolucion del sistema de monitoreo frente a la caída del maestro.

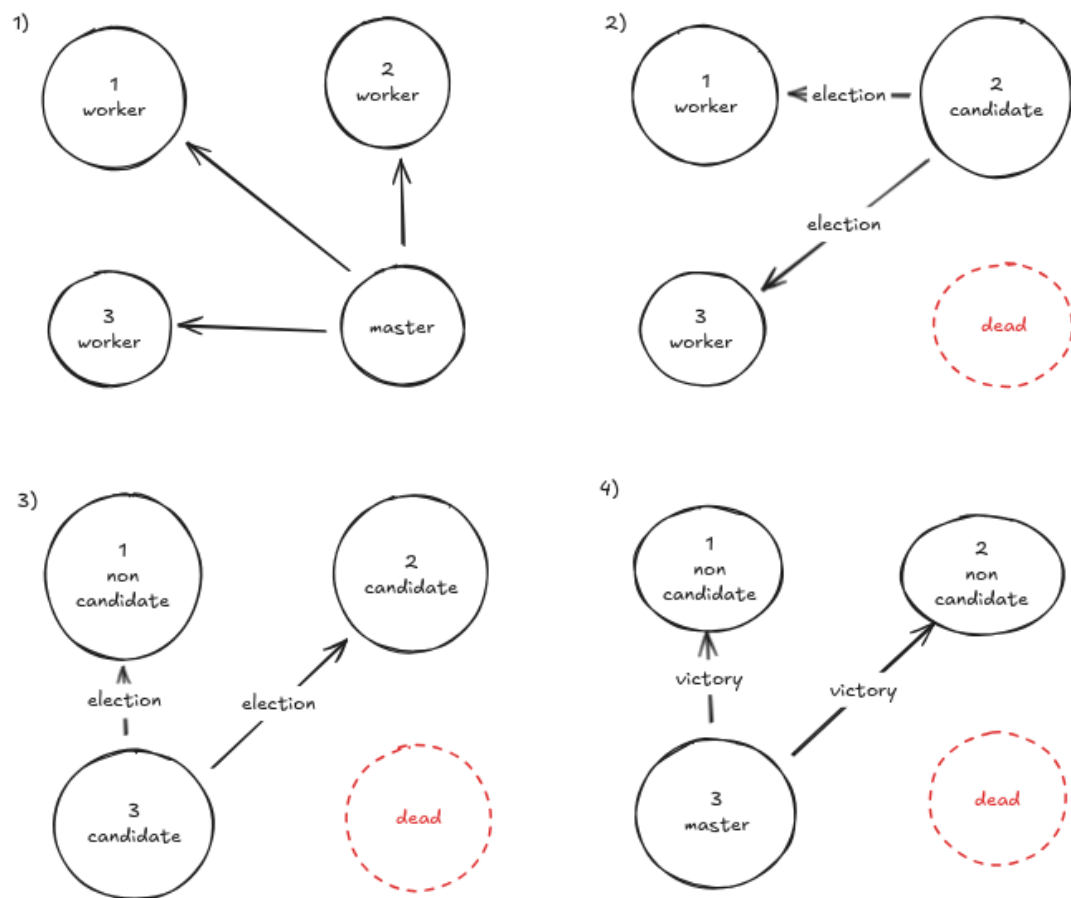


Figura 10: Seguimiento del Algoritmo "Bully"

6. Vista de Desarrollo

6.1. División del Sistema

Por último en esta sección se muestra como el sistema esta ordenado a nivel paquetes. En la figura 11 se puede ver el diagrama de paquetes mostrando la estructura interna del sistema y el cliente.

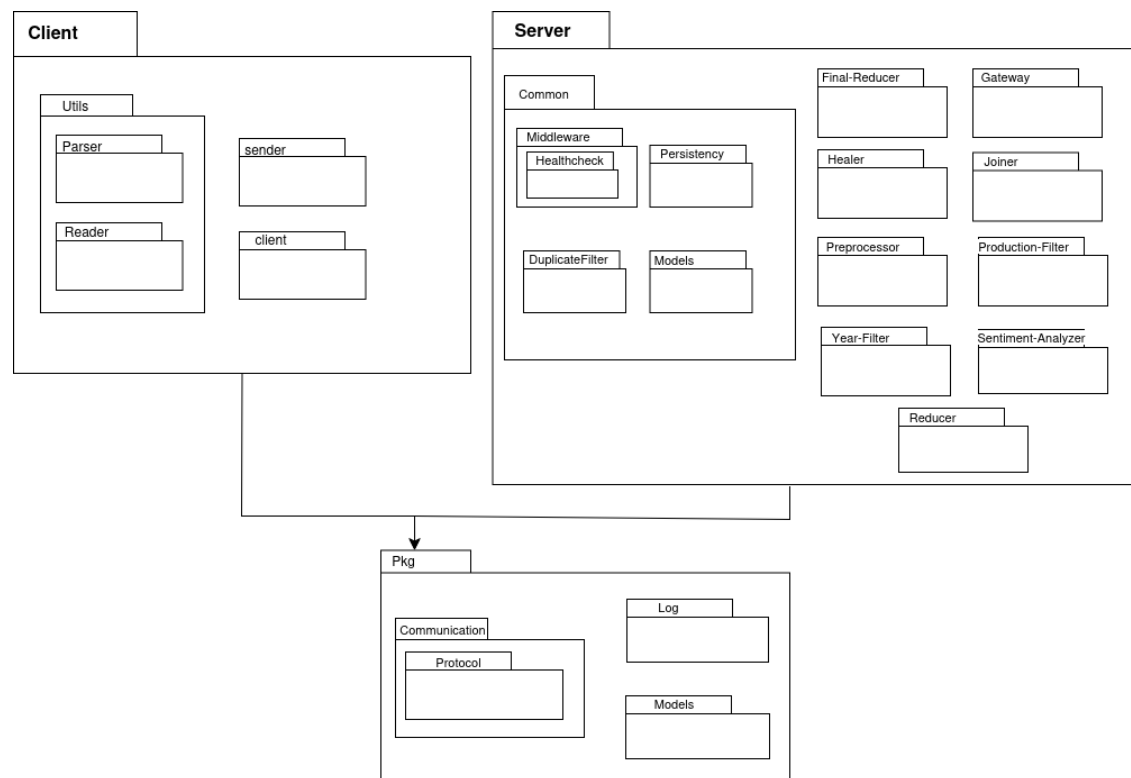


Figura 11: Diagrama de Paquetes

Tanto el cliente como el sistema (servidor) comparten una dependencia común ubicada en el paquete pkg, el cual incluye componentes reutilizables como:

- **Communication:** definiciones del protocolo de comunicación entre cliente y servidor
- **Log:** logging que comparte el cliente y servidor.
- **Models:** DTOs usados en comunicacion por el gateway, preprocessor y el cliente.

Además, dentro del servidor, todos los nodos funcionales (joiner, healer, year-filter, etc.) comparten una carpeta llamada common, que contiene:

- **Middleware:** encargado de manejar la comunicación y coordinación interna entre los nodos.
- **Modelos de datos:** definiciones compartidas como structs o DTOs.
- **Manejo de persistencia:** donde se encuentra el sistema de logs y snapshots para la tolerancia a fallas. También se encuentra la escritura segura archivos.

- **Manejo de duplicados:** paquete auxiliar para la lógica de filtrado de duplicados por ids incrementales con la ventana mencionada anteriormente.

En cuanto al cliente, además de la lógica de conexión y comunicación con el sistema, incluye un paquete utils, donde se ubican funciones auxiliares como la lectura y parseo de archivos de entrada.

7. Limitaciones y posibles mejoras

Aunque el proyecto actual cumple con los objetivos, existen áreas de mejora potencial:

- **Compresión del archivo de Logs:** permitiría reducir el uso de espacio en disco y acelerar la recuperación.
- **Formato binario para los Checkpoints:** una alternativa como `protobuf` o `MessagePack` podría mejorar la eficiencia sin perder portabilidad.
- **Checkpoints por cliente:** actualmente el sistema almacena un único Checkpoint por nodo, que contiene el estado de todos los clientes activos. Se podría segmentar este archivo y mantener un Checkpoint individual por cliente, almacenado en una carpeta dedicada. Esta estrategia tendría varias ventajas: permitiría una recuperación más granular y rápida (sólo se recarga el estado de los clientes necesarios), reduciría el impacto de corrupción o fallos en el archivo global, y facilitaría futuras extensiones como replicación selectiva o migración parcial de estado entre nodos.
- **Agrupamiento de batches vacíos:** en la implementación actual, los batches cuyo contenido ha sido completamente filtrado continúan siendo enviados como mensajes normales, ya que el peso (*weight*) de dichos batches sigue siendo relevante para el cálculo global. Sin embargo, este comportamiento puede optimizarse agrupando múltiples batches vacíos en un solo mensaje cuando la cantidad de datos útiles es insignificante. Esto permitiría reducir la carga de mensajes transmitidos, mejorar el rendimiento y aprovechar mejor la capacidad de cómputo de nodos altamente escalables.
- **Optimización de la información almacenada en Logs:** si bien la lógica de logeo es correcta, existen oportunidades para reducir la redundancia en el contenido registrado.